

Gli ordinali sono insiemi che permettono di misurare e confrontare insiemi ben ordinati. Un sistema è ordinale se è transitivo e ben fondato.

↳ Gli ordinali successivi sono ottenuti aggiungendo l'insieme stesso come nuovo elemento.

↳ Gli ordinali limite sono ordinali che non hanno predecessori immediati.

- La somma di ordinali consiste nel concatenare due insiemi ordinati

- La moltiplicazione di ordinali consiste nel ripetere l'insieme ordinato secondo l'ordine lessicografico.

- Né la somma né la moltiplicazione sono commutative per gli ordinali in generale.

ALFABETI e LINGUAGGI

- Simbolo $\rightarrow$ entità astratta

- Stringa $\rightarrow$ sequenza finita di simboli

- Lunghezza di una stringa $\rightarrow$ numero di simboli che contiene

↳ Un alfabeto Σ è un insieme finito di simboli

↳ Un linguaggio è un insieme di stringhe definite su un'alfabeto

OPERAZIONI su STRINGHE

- Concatenazione: combinare due stringhe una dopo l'altra.
 - Prefissi: sottoinsiemi iniziali di una stringa
 - Suffixi: sottoinsiemi finali di una stringa
 - Sotstringhe: sottoinsiemi continui di una stringa.
- ↳ $\epsilon \rightarrow$ stringa vuota, non modifica la stringa
- Σ^* è numerabile e include tutte le stringhe costruibili con i simboli Σ

AUTOMI

DFA

↳ Modello automatico per riconoscere linguaggi regolari.

↳ Definito da una quintupla $(Q, \Sigma, \delta, q_0, F)$

FUNZIONAMENTO

- Legge una stringa simbolo per simbolo e cambia stato in base alla funzione di transizione δ .
- Se termina in uno stato finale, la stringa è accettata.

LINGUAGGI REGOLARI

- Un linguaggio è regolare se esiste un automa che lo riconosce
- Descrivibili anche con espressioni regolari.

ESPRESSIONI REGOLARI

↳ L^* : tutte le concatenazioni di L , inclusa la stringa vuota $L \rightarrow$ linguaggio

↳ L^+ : tutte le concatenazioni di elementi di L , escludendo la stringa vuota.

- $\emptyset$ denota l'insieme vuoto: $L(\emptyset) = \emptyset$
- ϵ denota l'insieme che contiene solo la stringa vuota: $L(\epsilon) = \{\epsilon\}$
- $r+s$ denota l'unione dei linguaggi: $L(rs) = L(r) \cup L(s)$
- rs : denota la concatenazione dei linguaggi: $L(rs) = \{xy \mid x \in L(r), y \in L(s)\}$
- r^* : denota la chiusura di Kleene: $L(r^*) = \bigcup_{i=0}^{\infty} L(r)^i$

NOTAZIONE

- ↳ $r+s$: unione
- ↳ rs : concatenazione
- ↳ r^* : chiusura di Kleene
- ↳ r^+ : chiusura positiva

PROBLEMA della DECIDIBILITA'

↳ Verificare se una stringa x appartiene a un linguaggio L

↳ Decidibile se L è descritto da un DFA o da un'espressione regolare

Teorema Vuoto-infinito

- Non vuoto: un DFA ha un linguaggio non vuoto se e solo se accetta una stringa di lunghezza inferiore a n
- Infinito: Un DFA ha un linguaggio infinito se e solo se accetta una stringa di lunghezza tra n e $2n$
- Corollario: I problemi "linguaggio vuoto" e "linguaggio infinito" per un DFA sono decidibili.

Equivalenza

- Stabilire se i linguaggi accettati da due DFA sono uguali
- Decidibile costruendo un nuovo DFA per l'uguaglianza e verificando se il linguaggio risultante è vuoto

RELAZIONE di EQUIVALENZA

Partiziona un insieme in classi disgiunte di elementi equivalenti

RELAZIONE ASSOCIATA A UN LINGUAGGIO REGOLARE

Due stringhe sono equivalenti se, concatenando la stessa stringa a entrambe, si ottengono stringhe che appartengono o non appartengono al linguaggio nello stesso modo

Teorema di Myhill-Nerode

↳ Un linguaggio è regolare se e solo se esiste una relazione di equivalenza con un numero finito di classi.

↳ Le classi di equivalenza di questa relazione corrispondono agli stati del DFA che accetta il linguaggio.

Teorema 5.18

↳ Per ogni linguaggio regolare L , esiste un automa minimo che accetta L con il numero minimo di stati.

↳ L'automa minimo è unico a meno di isomorfismo (differenze nei nomi degli stati)

Dimostrazione

- La relazione di equivalenza R_M associata al DFA M raffina la relazione R_L . Il DFA costruito a partire da R_L ha il minimo numero di stati.
- Gli automi M e M' (minimo) sono isomorfi: esiste una funzione f che mappa correttamente gli stati e le transizioni.

ALGORITMO DI MINIMIZZAZIONE

1) Inizializzazione:

- Partiziona gli stati in finali F e non finali $Q - F$

2. Ciclo di partizionamento

- Per ogni insieme di stati, suddividili ulteriormente se gli stati reagiscono in modo diverso alle transizioni.
- Ripeti finché la partizione non cambia più.

3) Costruzione del DFA minimo

- Ogni insieme della partizione finale diventa uno stato del DFA minimo
- Definisci le transizioni del DFA minimo basandoti sui rappresentanti degli insiemi partizionati

4. Stati finali e stato iniziale

- Gli stati finali del DFA minimo corrispondono agli insiemi che contengono stati finali originali
- Lo stato iniziale è quello corrispondente allo stato iniziale del DFA originale.

5 Pulizia

- Elimina gli stati inutili senza archi entranti e gli archi associati.

GRAMMATICA LIBERA dal CONTESTO

$$G = \langle V, T, P, S \rangle$$

Ove V è l'insieme dei simboli non terminati, T l'insieme dei terminali, P l'insieme delle regole di produzione, e S il simbolo iniziale.

REGOLE di PRODUZIONE

Hanno la forma $A \xrightarrow{\text{PRODUCE}} \alpha$, dove A è un simbolo non terminale e α è una stringa di simboli terminali o non terminali.

Derivazioni

$\xRightarrow{G}$ Derivazione immediata (una sola applicazione di una produzione)

$\xRightarrow{G^i}$ Derivazione in i passi

$\xRightarrow{G^*}$ Derivazione in un numero qualsiasi di passi (zero o più)

Linguaggio generato da una grammatica

↳ $L(G)$ è l'insieme delle stringhe terminali che possono essere derivate dal simbolo iniziale S .

Linguaggio libero dal contesto

↳ Un linguaggio è libero dal contesto (CF) se esiste una grammatica che lo genera

Albero di derivazione

- ↳ Visualizza il processo di derivazione di una stringa.
- ↳ I nodi interni rappresentano simboli non terminati
- ↳ Le foglie rappresentano simboli terminati o ϵ (stringa vuota)

Proprietà dell'albero

- ↳ Radice etichetta con un simbolo non terminale
- ↳ I figli di un nodo etichettato A devono rispettare una regola di produzione $A \rightarrow X_1 X_2 \dots X_n$

Teorema 6.8

Una stringa α può essere derivata dal simbolo iniziale S se e solo se esiste un albero di derivazione che rappresenta α

Derivazione sinistra e destra

- Sinistra: La produzione viene applicata al simbolo non terminale più a sinistra.
- Destra: " più a destra

Ambiguità

- ↳ Una grammatica è ambigua se esiste almeno una stringa che ha più di un albero di derivazione.
- ↳ Un linguaggio è interamente ambiguo se ogni grammatica che lo genera è ambigua

FORMA NORMALE di CHOMSKY

- Le produzioni sono nella forma $A \rightarrow BC$ (due variabili) o $A \rightarrow a$ (un simbolo terminale)

- Forma normale di Greibach

- Le produzioni sono nella forma $A \rightarrow a\alpha$, dove a è un simbolo terminale e α è una stringa di variabili

- Eliminazione dei simboli inutili

- Fase 1: Elimina le variabili che non possono generare stringhe di terminali

- Fase 2: Elimina le variabili che non possono essere raggiunte dal simbolo iniziale S .

Teorema 6.12

Ogni linguaggio CF non vuoto può essere generato da una grammatica CF senza simboli inutili.

ELIMINAZIONE delle ϵ -produzioni

- Identificare le variabili annullabili (che possono derivare ϵ)

- Modificare le produzioni per eliminare le ϵ -produzioni, tranne eventualmente $S \rightarrow \epsilon$ se ϵ è nel linguaggio.

Teorema 6.14

- Ogni linguaggio CF può essere generato da una grammatica senza ϵ -produzioni (tranne eventualmente ϵ)

Eliminazione delle produzioni unitarie

- Identificare le coppie $A \xRightarrow{G} B$
- Sostituire le produzioni unitarie $A \rightarrow B$ con produzioni $A \rightarrow \beta$, dove $B \rightarrow \beta$ è una produzione non unitaria.

Teorema 6.15

Ogni grammatica CF senza ϵ può essere trasformata in una grammatica senza ϵ -produzioni, senza simboli inutili e senza produzioni unitarie.

Teorema di Chomsky

Ogni linguaggio CF senza ϵ può essere generato da una grammatica in cui le produzioni sono della forma $A \rightarrow BC$ o $A \rightarrow a$

Passaggi della trasformazione

- Elimina ϵ -produzioni, simboli inutili e produzioni unitarie.
- Riduci le produzioni con più di due simboli, introducendo nuove variabili
- Trasforma la grammatica finale nella forma normale di Chomsky

- ↳ Unfolding ed eliminazione della ricorsione sinistra garantiscono che tutte le produzioni partano da un simbolo terminale, eliminando potenziali ricorsioni infinite.
- ↳ La forma normale di Greibach è particolarmente utile nei parser di linguaggi di programmazione, dove è necessario gestire le produzioni da sinistra a destra in modo chiaro e univoco.
- ↳ La procedura garantisce che tutte le produzioni abbiano una forma semplice e uniforme, migliorando l'analisi sintattica dei linguaggi.

AUTOMI A PILA

Usa la pila LIFO per memorizzare simboli ed è più potente perché può riconoscere linguaggi più complessi

AUTOMA A PILA NON DETERMINISTICO

↳ Riconosce linguaggi usando stati, un nastro di input e una pila.

↳ Accetta stringhe in due modi: pila vuota o stato finale.

DETERMINISTICO VS NON DETERMINISTICO

I non deterministici sono più potenti perché esplorano più possibilità contemporaneamente.

Pumping lemma per linguaggi CF

- Linguaggio L CF $\rightarrow$ stringa lunga abbastanza $z = uvwx^i y$
- Le condizioni $|vx| \geq 1$, $|vwx| \leq n$, $uv^iwx^i y \in L$ per ogni $i \geq 0$

CHIUSURA dei LINGUAGGI CF

↳ Unione, concatenazione e chiusura di Kleene

Teorema 8.2

Una grammatica lineare destra genera un linguaggio regolare. $A \rightarrow aB$

Teorema 8.3

Ogni linguaggio regolare può essere generato da una grammatica lineare destra.

Le grammatiche lineari destra/sinistra sono strettamente legate ai linguaggi regolari

Grammatica di tipo 0

↳ Le più generali, generano linguaggi ricorsivamente enumerabili

Teorema 8.4

↳ Un linguaggio generato da una grammatica di tipo 0 è ricorsivamente enumerabile

Grammatiche di tipo 1

↳ Produzioni con regole di contesto, generano linguaggi ricorsivi.

Teorema 8.5

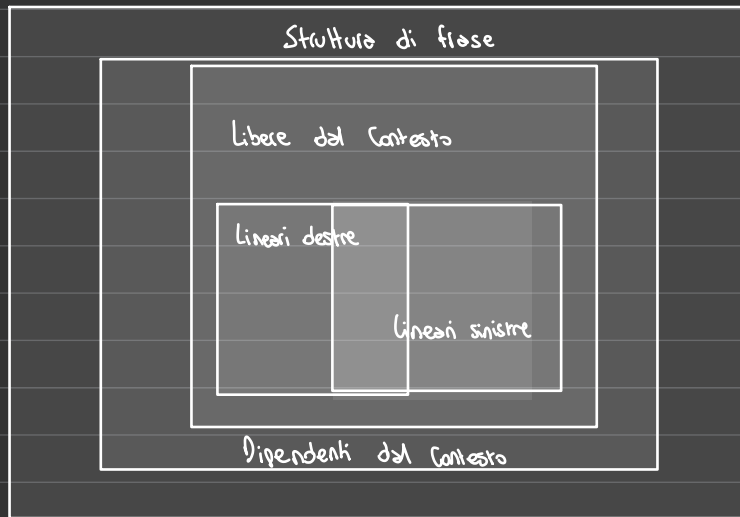
↳ Ogni linguaggio generato da una grammatica di tipo 1 è ricorsivo

Teorema 8.6

↳ Non tutti i linguaggi ricorsivi sono generati da grammatiche di tipo 1

- Funzione 0, successore, proiezione i -esima: calcolate da prammatiche regolari
- Funzioni $+$ e $-$: calcolate da prammatiche libere del contesto
- Funzione moltiplicazione $f(x,y) = xy$: calcolata da una grammatica di tipo 1

Gerarchia di Chomsky



CALCOLABILITÀ

Algoritmo: Funzione che associa un input a un output

Funzione totale: Definita per ogni input; l'algoritmo termina sempre

Problema della funzione $h(x) = g_x(x) + 1$: Dimostra che non tutte le funzioni sono calcolabili in un sistema finito.

Programma: Estensione degli algoritmi che può non terminare su alcun input, permettendo esecuzioni infinite.

MACCHINA DI TURING

↳ Modello ideale di calcolabilità

↳ Componenti: nastro infinito, testina di lettura/scrittura, controllo a stati finiti

↳ Funzionamento: Computazione tramite una matrice funzionale che definisce le azioni in base allo stato e al simbolo letto.

↳ Caratteristiche: deterministica, memoria infinita, opera in modo discreto, possibilità di non terminare

Funzione Turing calcolabile: Una funzione calcolata da una MDT che termina con il risultato scritto sul nastro.

Codifica unitaria: I numeri naturali sono rappresentati da una sequenza di simboli "1" consecutivi

Funzione parziale: Una funzione che non è definita per tutti gli input, quindi la MDT può non terminare

Insiemi ricorsivamente enumerabili

Un insieme A è ricorsivamente enumerabile se esiste:

1. Una funzione parziale ricorsiva $\varphi(x)$ che riconosce gli elementi di A
2. Un programma (o MT) che può:

- ↳ Terminare e restituire 1 per i numeri $x \in A$
- ↳ Non terminare mai (o divergere) per i numeri $x \notin A$

Funzione semi-caratteristica

La funzione semi-caratteristica per un insieme A :

$$A(x) = \begin{cases} 1 & \text{se } x \in A \\ \text{non termina} & \text{se } x \notin A \end{cases}$$

Dovetailing

La tecnica del dovetailing è utilizzata per esplorare più computazioni in parallelo, passo per passo, evitando di bloccarsi in un loop infinito su un singolo input. È spesso applicata per verificare se una MT termina su almeno un input.

Insiemi ricorsivi

Un insieme A è ricorsivo se possiamo determinare, per ogni elemento, in un tempo finito:

- Se $x \in A$
- Se $x \notin A$

Dimostrare che un sistema è re

1. Costruzione della funzione semi-caratteristica:

- ↳ Scrivere un algoritmo (pseudo-codice) che:
 - Termina restituendo 1 per $x \in A$
 - Diverge per $x \notin A$
- ↳ Spesso si costruisce utilizzando una MdT specifica

2. Dimostrazione come range di una funzione parziale ricorsiva

- Definire una funzione f che:
 - ↳ Ha come range gli elementi di A
- Mostrare che f è calcolabile (ricorsiva parziale)

3. Analisi passo-passo del dominio:

- Dimostrare che $A = \text{dom}(\varphi)$, cioè A è esattamente l'insieme degli input su cui la funzione φ termina.

Dimostrare che un insieme è non re

1. Teorema di Post

- Se A non è ricorsivo e A^c (il complementare) è re, allora A non può essere re

2. Riduzione a un problema indecidibile:

- Mostrare che la definizione di A implica decidere un problema indecidibile

3. Dimostrare che il complementare NON è re:

- Provare che non esiste una MdT che possa enumerare gli elementi del complementare A^c .

Tesi di Church-Turing

- Ogni funzione calcolabile è calcolabile da una MdT
- Ogni modello di calcolo equivalente (λ -calcolo, automi, linguaggi di programmazione) è riconducibile alla MdT

Problema della Fermata

- Formalizzazione: non esiste una MdT H che, data una MdT M e un input w , determini sempre se $M(w)$ termina.
- $H(M, w)$ = indecidibile

Aritmetizzazione

- Codifica della MdT come numero $e(M)$: uno schema per rappresentare stati, regole e alfabeto usando numeri naturali

Teorema di Incompletezza di Gödel

- ↳ Prima legge: Un sistema formale coerente non può essere completo
- ↳ Seconda legge: Un sistema coerente non può dimostrare la propria coerenza



Teorema di Rice

- Ogni proprietà non banale dei linguaggi accettati dalle MdT è indecidibile.

Proprietà non banale

- Vale per alcuni linguaggi, ma non per tutti

ESEMPLI PROPRIETÀ INDECIDIBILI

- Linguaggio vuoto ($L(M) = \emptyset$)
- Linguaggio finito
- Linguaggio che contiene una stringa w

Proprietà banali

Valgono per tutti o nessuno. $P(L(M)) =$ sempre vero o sempre falso

Dimostrazione Teorema di RICE

• PROBLEMA della fermata: Non esiste un algoritmo che decida sempre se una MdT termina su un dato input.

1. Supponiamo che P sia decidibile
2. Costruiamo M'' per ridurre il Problema della Fermata a P
3. Contraddizione: il Problema della fermata è indecidibile, quindi P è indecidibile

Dimostrare che un insieme è creativo

- Verificare che A sia re $\rightarrow$ Costruire una MdT che enumera A
- Dimostrare che A^c non è re $\rightarrow$ Usare una riduzione da un problema indecidibile
- Costruire una funzione produttiva $\rightarrow$ Mostrare che per ogni $W \subseteq A^c$, trovi un elemento in A

Dimostrare che è produttivo

B è produttivo se ha una funzione $f(w)$ che trova nuovi elementi in B

- Mostrare che f genera elementi $x \in B$ non in w
- Dimostrare che B^c non può essere enumerato
- Se B^c è creativo, allora B è produttivo